

RFLCS — Repetition-Free Longest Common Subsequence

This project implements exact and heuristic algorithms for solving the **Repetition-Free Longest Common Subsequence (RFLCS)** problem. It uses Gurobi as the ILP backend.

Requirements

- **C++ compiler** supporting **C++23**, tested with:
 - g++ (GCC) 15.2.1 20250813
 - clang++ 20.1.8
 - Intel(R) oneAPI DPC++/C++ Compiler 2025.2.1
 - **CMake 3.22.1**
 - **Gurobi Optimizer v12.02** It is free to use for academic purposes. An installation guide is provided under How do I install Gurobi Optimizer?. And a guide for license setup is available under How do I retrieve and set up a Gurobi license? Older C++ compilers, such as g++-11, require the workaround described in How do I resolve "undefined reference" errors while linking Gurobi in C++?. Unfortunately, in our experience this workaround is not always reliable.
 - **python3**
 - **pdflatex**
-

Platform Compatibility

Tested on:

- Linux (Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-59-generic x86_64))
 - macOS (Apple Silicon M3 macOS 15.5:24F74)
-

Environment Variables

Before building, make sure the following environment variable is set (included in the installation guide) and cmake can access it:

Variable	Description	Examples
GUROBI_HOME	Path to Gurobi installation	/opt/gurobi1200/linux64 or /Library/gurobi1201/macos_universal2

Build Instructions

The project is built with cmake

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
cp build/rflcs ./rflcs
```

Run instructions

The executable takes one parameter `-i` pointing to the input file. E.g. one instance

```
./rflcs -i RFLCS_instances/type1/512_8reps.24
```

or all instances (takes 120+ hours)

```
find ./RFLCS_instances/**/* -type f -exec ./rflcs -i {} \;
```

or all solvable instances (takes <1h)

```
find ./RFLCS_instances/type0/{32,64,128,256,512}* -type f -exec ./rflcs -i {} \;
find ./RFLCS_instances/type0/{1024,2048,4096}* \
  -type f ! -name '*_n-div-8.*' \
  -exec ./rflcs -i {} \;
find ./RFLCS_instances/type1/* -type f -exec ./rflcs -i {} \;
```

The `runme.sh` script prompts the user to choose which set of experiment instances to run:

1. **All (>120h)** – Runs experiments for all instance complexity categories.
2. **Solvable (<1h)** – Runs experiments only for solvable instance complexity categories.

If the user does not provide any input within **60 seconds**, the script automatically defaults to **All (>120h)**. Invalid input is also treated as the default choice.

Result Files

The result files from our original experiments are provided in the `experiments` folder.

Result File Example

Each experiment generates a text file in the `results` folder with key metrics. Below is an example, with a brief description for each field:

Field	Example Value	Description
solved	true	Whether the algorithm found and proved an optimal solution
solution_length	8	Length of the solution found
upper_bound	8	Derived upper bound
solution_runtime	8.8e-05	Time taken to find the solution (in seconds)
heuristic_solution_length	8	Length of the solution found by the heuristic
heuristic_solution_runtime	6.4e-05	Time taken by the heuristic (in seconds)
heuristic_runtime	8.3e-05	Total runtime of the heuristic (in seconds)
solving_forward	true	Indicates forward-solving was applied
reduction_is_complete	true	Whether the MDD reduction process completed for all characters
reduction_quality	1	Quality of the reduction (float between 0 and 1)
reduction_upper_bound	8	Upper bound after the MDD reduction
reduction_runtime	5e-06	Time taken for the MDD reduction process
mdd_memory_consumption	0	Memory used by the MDD (bytes)
main_process_memory_consumption	0	Memory used by the main process (bytes)
solver_solution_length	0	Length of the solution found by the ILP
solver_upper_bound	0	Upper bound reported by the ILP
solver_runtime	0	Time taken by the ILP
solution	[20, 12, 23, 15, 1, 10, 22, 16]	The found solution subsequence

Experiment Evaluation

With the result files, the comparison result tables from the paper can be generated.

```
python3 -m venv .venv
source .venv/bin/activate
```

```
python3 -m pip install pandas seaborn matplotlib
cd results || exit
python3 latex_table.py
python3 box_plots.py
deactivate
pdflatex -jobname=results main.tex
mv results.pdf ../results.pdf
rm ./*.aux
rm ./*.log
rm ./*.pdf
rm ./table_*.tex
cd ..
```

The results.pdf file contains the result tables and box plots generated from the experiments. Any experiments that were not executed will not appear in these artifacts.

Dependencies

This project includes third-party libraries (under external/) under the following licenses:

- Boost — Boost Software License 1.0
- Abseil — Apache License 2.0

These are compatible with the GPLv3 license under which this project is released.